wsdm-cup-multilingual-chatbot-arena

4位 Solution

Rank	4
Team	HKUST-GZ DSA KiMi Lab
Author	HengweiDAI
Topic ID	568522
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/568522
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

🙇🙇

コミュニティの仲間の皆さんの誠実な共有に感謝します。特に、前回のLMSYSコンペでpipelineを共有してくれた @sayoulala に感謝します。情熱あるすべての参加者に敬意を表し、栄誉を勝ち取った入賞者の皆さんにお祝いを申し上げます。また、**40*A100 80GB GPUの支援！**を提供してくださった香港科技大学（広州）DSA ThrustのKIMI Labにも特別な感謝を捧げます。@HKUST-gz

私のsolutionは、特に他の優秀な参加者と比べると比較的シンプルかもしれません。しかし、ソロで金メダルを取ったのはこれが初めてです。栄誉を得た喜びに比べると、コンペ中の挫折や困難が90%を占めていました。遠慮なくご指導いただければ幸いです。ありがとうございます！

目次

- TL;DR
- Solutionの詳細
  - 出力AとBを制御する初期promptとしてのCoT
  - AutoModelForSequenceClassification ——> AutoModelForCasualLM
  - Post-Pretrainの改良
  - 学習lossを段階的に設計する理由
  - vllmでallowed_token_ids=[a_tok_id,b_tok_id]とlogprobs=Nを併用して特定tokenのlogitを得る
  - 最終的な量子化方式としてGPTQ 8-bitを使う
- いくつかの失敗した試み
- 実現しなかったアイデア
- まとめ

TL;DR

1. AutoModelForCasualLM

(1) AutoModelForSequenceClassification + transformersによる推論を、AutoModelForCasualLM + vllmによる推論に置き換えます。

(2) Token AとToken Bのlogitを比較して出力を決めます。

(3) Chain-of-Thought（CoT）を初期promptとして使います。

2. Post-PretrainとFinetuneの違い

(1) Datasetの違い: ultrafeedback + C4AI-Community/multilingual-reward-bench（Post-Pretrain用）、lmsys（tieとラベル付けされたデータを除く）+ wsdm（Finetune用）。

(2) Loss計算の違い: pretrainingでは、語彙全体に対するAまたはBのcross-entropy lossを使い、finetuningではAとBの間のcross-entropy lossを使います。

(3) Data Augmentationの違い: pretrainingでは、同じbatch内でresponseA+BとresponseB+Aを使い、finetuningではresponseA+Bだけを使います。

(4) 入力長の違い: pretrainingでは1024 token、finetuningでは2048 tokenを使います。

3. Distillationの最適化

(1) 同じ手順で、Athene-v2-chat + nvidia/Llama-3.1-Nemotron-70B-Instruct-HF + Qwen2.5-72B-Instructをfine-tuning datasetでそれぞれ学習し、soft labelを生成します。

(2) finetuningでは1 epochより多く、具体的には2 epoch学習します。cross-validationの結果は、2 epoch目の終盤に向けて大きく改善します（平均約0.002の改善）。

4. 推論の最適化

(1) vllmでallowed_token_ids=[a_tok_id,b_tok_id]とlogprobs=Nを併用して、特定tokenのlogitを得ます。

(2) awqをgptqに置き換えます。

Solutionの詳細

出力AとBを制御する初期promptとしてのCoT

```
():
    messages = [
    {
       :,
       :
    },
    {
       :,
       :
    }
  ]
    text = tokenizer.apply_chat_template(messages, tokenize=, add_generation_prompt=)
     text+
```

AutoModelForSequenceClassification ——> AutoModelForCasualLM

- tokenizerを変更します

```
():
    tokenizer = AutoTokenizer.from_pretrained(
        path,
        add_eos_token=,)
    tokenizer.padding_side =
     tokenizer
```

- finetuningでは、Token AとToken Bのlogitだけを使って二値分類のlossを計算します
- ラベルの対応：A ——> 0, B ——> 1

```
(nn.Module):
  ():
        ().__init__()
        .model = base_model
        .token_ids = []
         letter  [,]:
            token_id = tokenizer(letter, add_special_tokens=)[][-]
            .tok_locations.append(token_id)
  ():
        outputs = .model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=)
        scores = []
         token_id  .tok_locations:
            score = outputs.logits[:, -, token_id]
            scores.append(score)
        logits = torch.stack(scores, )
         logits.contiguous()
  ():
        logits = .encode(input_ids, attention_mask)
        ce_loss = (.loss_fn(logits, labels)).mean()
```

Post-Pretrainの改良：

- 語彙全体に対するToken AとToken Bのlogitを使ってcross-entropy lossを計算します

```
(nn.Module):
  ():
        ().__init__()
        .model = base_model
        .token_ids = []
         letter  [,]:
            token_id = tokenizer(letter, add_special_tokens=)[][-]
            .tok_locations.append(token_id)
  ():
        outputs = .model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=)
        logits = outputs.logits[:, -, :]
        ce_loss = (.loss_fn(logits, labels)).mean()
```

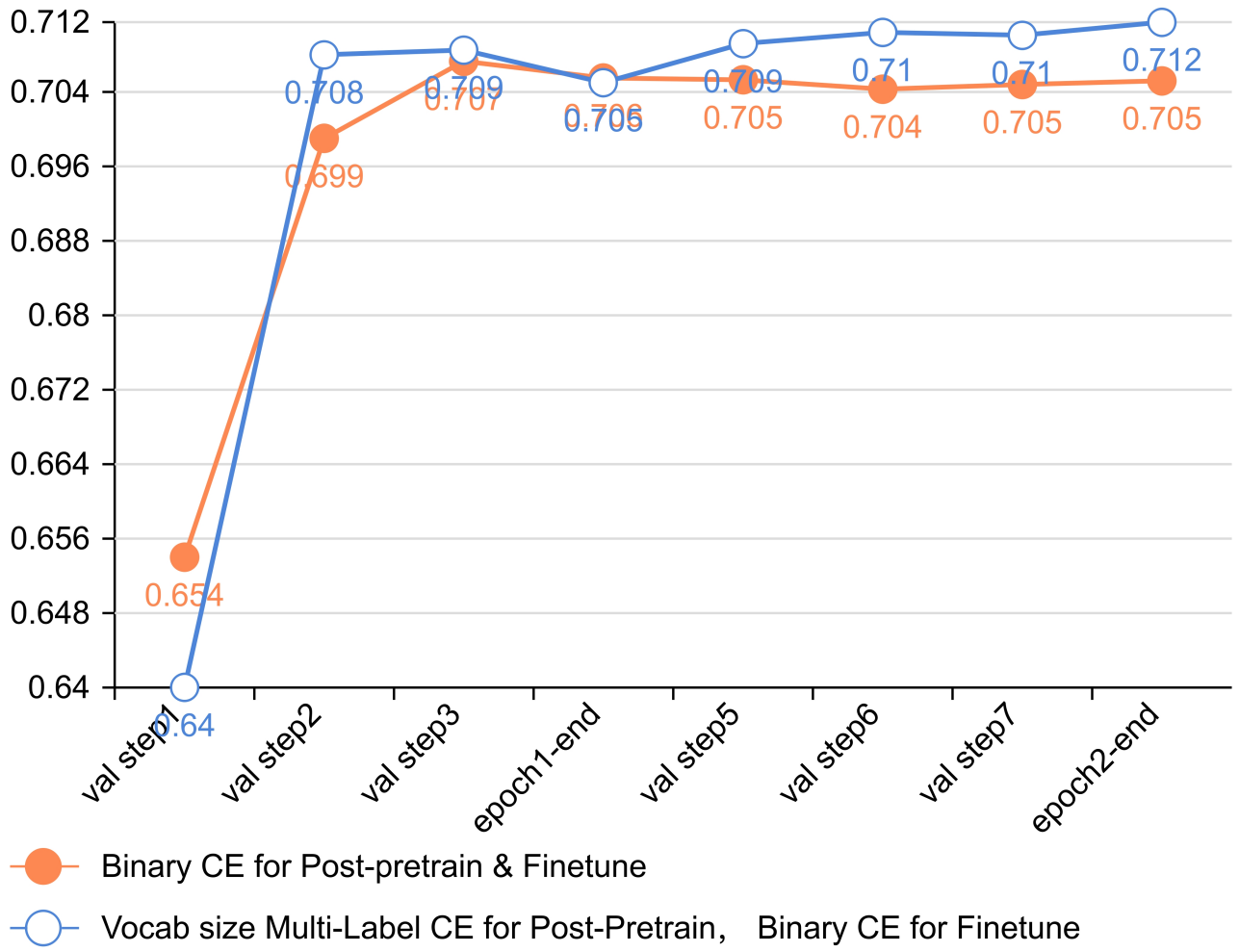
- ラベルの対応：0 ——> A_token_id, 1——> B_token_id
- 同じbatch内でサンプルを反転させてdata augmentationを行います

```
():
  () -> :
        ().__init__()
    ......
        .tokenizer = tokenizer
        .a = tokenizer.encode()[]
        .b = tokenizer.encode()[]
  ():
        _, data = data_entry
        text = data[]
        text2 = data[]
        features = .tokenizer(text,padding=,add_special_tokens=,return_length=)
        features2 = .tokenizer(text2,padding=,add_special_tokens=,return_length=)
        labels = .a  data[]==  .b
        labels2 = .a  data[]==  .b
         features[],features[],features[], labels,features2[],features2[],labels2
    ......
```

学習lossを段階的に設計する理由

Post Pretrainでは語彙全体を使ってAとBの多クラスcross-entropyを計算し、Finetuneでは二値のcross-entropyを使います。この方法でmodel性能の上限を引き上げられますが、より多くの学習stepが必要になります。

fold	epoch 1のbest CV	epoch 2のbest CV
0	0.7184	0.7209
1	0.7108	0.7153
2	0.7134	0.7145
3	0.7044	0.7091
4	0.7085	0.7115



vllmでallowed_token_ids=[a_tok_id,b_tok_id]とlogprobs=Nを併用して特定tokenのlogitを得る

（logprobs=Nだけを使って上位N個のtokenを選ぶと、Token AもToken Bも上位N個に入らない可能性があります）

```
a_tok_id = tokenizer(, add_special_tokens=)[][-]
b_tok_id = tokenizer(, add_special_tokens=)[][-]
llm = vllm.LLM(
    cfg.model_dir,
    quantization=,
    tensor_parallel_size=,
……)
sampling_params = vllm.SamplingParams(n=, top_k=, logprobs=, max_tokens=, temperature=,
skip_special_tokens=, allowed_token_ids=[a_tok_id,b_tok_id])
responses = llm.generate(test[], sampling_params, use_tqdm=)
```

最終的な量子化方式としてGPTQ 8-bitを使う

float16 CV：0.714 GPTQ 8-bit量子化を使用: CV: 0.713, LB: 0.703, PB: 0.714 AWQ 4-bit量子化を使用: CV: 0.710, LB: 0.708, PB: 0.709

いくつかの失敗した試み

- **Rank model**
- **distillationに使うmodelの数を増やす:** Llama-3.3-70B-Instructまで学習を広げてみましたが、4つのmodelでdistillationした効果は3つのmodelの場合と大きな差はなく、差は0.0005を超えませんでした。
- **難しいサンプルのフィルタリング:** 難しいサンプルは主に2つの状況から生じます。modelの能力不足と、普遍的な価値観に反するサンプル固有のbiasです。明らかな誤りやbiasを持つ異常サンプルは、学習中にlossの振動を引き起こします。こうしたbiasは特定のユーザーのbiasに由来することが多く、サンプル集団全体の一般的な価値体系とは異なります。そこで、fine-tuningのOOF（Out-of-Fold）データで6つの70B modelを学習してsoft votingを行い、総合scoreと実際のラベルとの乖離が最も大きい上位5%〜10%のサンプルを取り除きました。残りのデータで14B modelをfine-tuneしましたが、これらのサンプルを取り除かない場合と比べて性能に有意な差はありませんでした。手法に改善の余地があるということかもしれません。

実現しなかったアイデア

- **複数modelの融合 > 単一modelのTTA（Test Time Augmentation）:** 複数modelの融合をテストしたところ、単一modelで713のscoreを達成でき、gemmaとの融合では715/716に達する可能性がありました。
- **動的なmodel選択 > 単一model:** まず、既存のプロセスと異なるLLMを使ってN個の個別modelを学習します。OOF（Out-of-Fold）データで、各質問に対する各modelの回答の信頼度を計算します。次に、Mixture of Experts（MOE）のrouterに似たmodel selectorを学習し、各質問を判断に最も信頼できるmodelへ割り当てます。各個別modelを別々にloadして、割り当てられたサンプルを推論させます（注: 推論時間を正確に制御するのは難しく、極端な場合、1つのmodelがすべてのサンプルを推論する必要が生じることもあります）。

推論コード：here